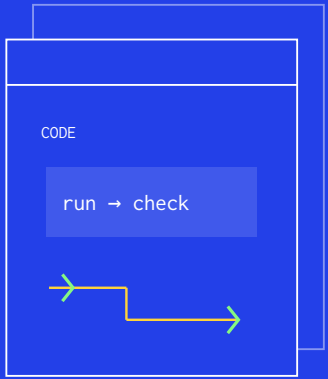
WEEK 2 / LAB 3 / TEAM WORK

MLOps: Machine Learning Operations

Same code.
Clean environment.

Lock the dependencies, package your baseline in a container you write, and rerun it away from your laptop. Needs Lab 2 merged on main.



Done means a teammate can rerun it without you

Three outcomes you can point to in the team repository.

1 **Locked**

Dependencies come from the committed uv.lock; a stale or missing lock fails visibly instead of being regenerated.

2 **Packaged**

A Dockerfile and .dockerignore you wrote build an image that runs the Lab 2 baseline and its tests.

3 **Rerun**

A fresh container reproduces the Lab 2 baseline from unchanged inputs; any difference is observed and explained.

Who does what

The split is yours; every member owns something and reviews something.

INDIVIDUAL

One PR each

One area: lock, Dockerfile, rerun or README. One branch, one PR.

TEAM

One baseline

Agree areas, merge order and the commit to package. Final rerun by a non-author.

REVIEW

Another member

Review a PR you did not write: rebuild from the README and leave a real finding.

Individual report reports/week-2.md covers Labs 3–4 and lives in your personal repository.

Add these to the team repository

Paths are the contract; contents are your design.

REQUIRED

Commit them on main through reviewed PRs. Nothing in this list may need your laptop's .venv, caches or secrets.

TEAM REPOSITORY PATHS

```
Dockerfile
.dockerignore
README.md
your team's own report
```

README: how to build and rerun. Report: your team chooses its format; link raw logs and the commit.

New here: a Dockerfile for a uv project

Lecture 3 showed FROM, COPY, RUN and CMD. Each line below is a decision you make and justify.

DECISIONS, NOT ANSWERS

The lock should drive the install, and the build context should hold only what the image needs. Write it, build it, then break it on purpose.

DOCKERFILE CHECKLIST

```
FROM pinned Python 3.12 + uv
COPY manifest + lock first - why?
RUN install exactly the lock
COPY code, config, data snapshot
CMD command that runs the baseline
.dockerignore: what never enters?
working directory: where?
```

References: Docker Dockerfile reference; uv documentation, Docker integration.

Example: a Dockerfile for a small uv project

A toy app called hello; your files, paths and command will differ.

ILLUSTRATIVE EXAMPLE

Start from a base image, copy the manifest and lock, install exactly the lock, then copy code. Add a WORKDIR first so files land in /app.

DOCKERFILE EXAMPLE

```
FROM python:3.12-slim
COPY --from=ghcr.io/astral-sh/uv:0.12 /uv /bin/
COPY pyproject.toml uv.lock ./
RUN uv sync --locked --no-install-project
COPY src ./src
RUN uv sync --locked
CMD ["uv", "run", "python", "-m", "hello"]
```

Different project on purpose: read it, then adapt it to yours; do not paste it.

Example: what a .dockerignore keeps out

One pattern per line; paths are relative to the build context.

ILLUSTRATIVE EXAMPLE

Anything listed is not sent to the build, so it cannot leak into the image or invalidate the cache.

.DOCKERIGNORE EXAMPLE

```
.venv
.git
__pycache__
.pytest_cache
artifacts
tracking
.env
```

Decide for your project: is any data or config needed inside the image?

Example: build, run and keep the outputs

The same commands work for any image; names are yours to choose.

ILLUSTRATIVE EXAMPLE

Check the lock first. Build once, run a named container, copy results out before you remove it.

TERMINAL EXAMPLE

```
uv lock --check
uv sync --locked
docker build -t hello:dev .
docker run --name rerun hello:dev
docker cp rerun:/app/out ./out
docker image inspect hello:dev
```

Do not use --rm on a run whose outputs you still need. Add --network none to prove nothing is downloaded.

Rules that keep the rerun honest

Fixed for every team; everything else is your choice.

1 **Clean build, no host state**

Build from a clean checkout; no bind-mounted source, .venv or earlier outputs.

2 **Unchanged inputs**

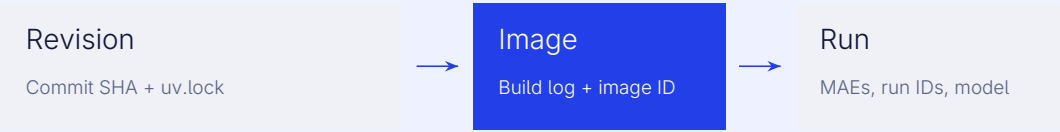
Same data snapshot, config and split as Lab 2. No tuning, API, cloud or test-set scoring.

3 **A stale lock must stop**

On a throwaway copy, show that a missing or stale lock fails. Never regenerate it to get green.

Link the result to what produced it

One chain from commit to observed metrics.



`A screenshot alone does not connect the three.`

Keep raw logs your report can link to; at most three screenshots. Compare with Lab 2 and explain differences.

Review, blockers and handover

What a reviewer checks and what an honest record contains.

REVIEW

Rebuild from README

Follow the README only; check ignore rules, locked install, evidence.

BLOCKED

Name the step

Failing step, exact message, next owner. Never hide a check.

HANDOVER

Merge, then Lab 4

Merge after review and checks. Lab 4 builds on this image.

Local runs are not remote CI; keep them as separate claims.

EXIT CHECK

Could a teammate rebuild it without asking you?

Next: Lab 4 serves the evaluated model through an API on this image. m2 is tagged after Lab 4.